

MI-Workbench: an autonomous multi-agent platform for mechanistic interpretability of biological foundation models

Ihor Kendiukhov^{1*}

¹ Department of Computer Science, University of Tuebingen, Tuebingen, Germany

* ihor.kendiukhov@student.uni-tuebingen.de

Abstract

Mechanistic interpretability (MI) of biological foundation models—such as single-cell transformers trained on gene expression data—requires iterative cycles of hypothesis formation, experimental execution, statistical evaluation, and adversarial critique. These cycles are labor-intensive, error-prone, and difficult to reproduce. We present MI-Workbench, an open-source platform that coordinates a team of large language model (LLM) agents through a single, inspectable orchestrator to carry out MI investigations. Workflows are declared in a graph-based domain-specific language (DSL) whose nodes are agent roles—executor, a configurable panel of reviewer lenses (rigour, adversarial, biological plausibility), idea generator, and knowledge extractor—and each agent is reached through a provider-agnostic adapter layer supporting Claude Code, OpenAI Codex, and Google Gemini. Key mechanisms include: (i) a consensus reviewer that runs a reviewer panel concurrently and merges their critiques by Jaccard deduplication with agreement-based severity escalation and an optional advancement gate; (ii) structured, artifact-aware feedback that maps critiques to specific artifacts and sections for targeted revision; (iii) optional sandboxed execution of the analysis code an agent emits, so reviewers critique executed results rather than an untested plan; and (iv) an output-cleaning pipeline, automated convergence detection, and reproducibility packaging. We characterise the orchestrator with deterministic ablation and load experiments—larger reviewer panels surface strictly more distinct, complementary

critiques, and the coordinator sustains concurrent runs with graceful tail-latency growth—and corroborate the panel with a live Gemini backend. A case study on whether Geneformer attention heads encode causal gene regulation shows the platform identifying key confounders (co-expression bias, rank-value encoding artifacts, attention symmetry) and refining analyses through structured feedback without human intervention. The system comprises roughly 9,200 lines of backend Python with an accompanying suite of over 370 tests, and supports real-time monitoring via WebSocket streaming.

Introduction

The rapid development of biological foundation models—including scGPT [1], Geneformer [2], and scBERT [3]—has raised fundamental questions about what these models learn and how their internal representations relate to known biology. Mechanistic interpretability (MI), the discipline of reverse-engineering the computational mechanisms within neural networks [4–6], offers a principled framework for answering these questions. However, applying MI to biological models presents unique challenges:

1. **Confounding structures.** Gene co-expression, regulatory network degree distributions, and expression magnitude biases create complex confound structures that require carefully controlled null models [7].
2. **Statistical rigor.** Claims about learned regulatory mechanisms require multiple testing correction, effect size reporting, confidence intervals, and proper null model comparison—standards often missing from initial analyses.
3. **Iterative refinement.** MI investigations are inherently cyclical: an initial analysis reveals confounders, which motivate new controls, which reveal subtler effects, and so on.
4. **Reproducibility.** Complex computational pipelines with many degrees of freedom are difficult to reproduce without meticulous documentation of parameters, data versions, and analytical choices.

While individual LLMs have demonstrated impressive capability for scientific reasoning [8,9], a single agent prompted once lacks the adversarial pressure and

structured quality control that rigorous research demands. A human researcher instead cycles between the roles of executor (performing analysis), reviewer (critiquing methodology), and devil’s advocate (searching for confounders and alternative explanations). MI-Workbench assigns these roles to distinct LLM agents and coordinates them.

The system is best described as an *agentic orchestration platform* rather than a distributed multi-agent system in the classical sense: agents do not run autonomous, asynchronous control loops or exchange messages through an agent communication language. A single orchestrator sequences them according to an explicit workflow graph. This centralization is a deliberate design choice for scientific quality assurance—it makes every run deterministic in its control flow, inspectable, auditable, and reproducible—and it does not preclude concurrency where it matters: the reviewer panel is evaluated in parallel within an iteration. We introduce MI-Workbench, a platform that automates this multi-role research process. Our key contributions are:

- A **graph-based workflow DSL** that lets researchers declare multi-role research loops with conditional execution, concurrent reviewer fan-out, and configurable stop conditions, validated before execution.
- A **consensus reviewer mechanism** that runs a configurable panel of critic lenses concurrently and merges their findings through deduplication, agreement-based severity escalation, deterministic ranking, and an optional advancement gate.
- A **structured feedback loop** that maps reviewer critiques to specific artifacts and sections, enabling targeted revision rather than wholesale rewriting.
- **Sandboxed verified execution** that optionally runs the analysis code an executor emits and folds the captured results back into review, so critique targets executed output rather than an untested plan.
- A **provider-agnostic adapter architecture** abstracting over Claude Code, OpenAI Codex, and Google Gemini CLI backends through a common interface, with an integrated **knowledge extraction** pipeline and end-to-end **reproducibility packaging**.

- A **quantitative evaluation** of the orchestrator—panel-size ablations, reviewer-lens complementarity, convergence behaviour, and concurrent-run load benchmarks—alongside a domain case study.

Related work

Autonomous scientific agents. The vision of AI-driven scientific discovery has been explored through systems like ChemCrow [10], which chains LLM reasoning with chemistry tools, and Coscientist [9], which demonstrated autonomous experimental design in chemistry. SciAgents [11] explored multi-agent scientific reasoning using graph-based ontologies. These systems focus primarily on wet-lab experimental design; MI-Workbench targets the distinct challenge of computational interpretability with its emphasis on statistical rigor and confound control.

Multi-agent LLM frameworks. Multi-agent debate and collaboration have been studied in frameworks such as CAMEL [12], AutoGen [13], and ChatDev [14]. MetaGPT [15] introduced structured output schemas for software engineering agents. CrewAI and LangGraph provide general-purpose agent orchestration. Classical multi-agent systems emphasise autonomous agents with independent control loops that coordinate through message passing; MI-Workbench instead adopts a centrally orchestrated design in which a single engine sequences role-specialised LLM calls along an explicit graph. We make this trade-off deliberately: for automated scientific critique, a deterministic and inspectable control flow is worth more than agent autonomy, because it is what makes a run auditable and reproducible. MI-Workbench also differs from these frameworks in its domain specialization—the agent roles, prompt templates, artifact types, and quality criteria are designed for mechanistic interpretability research—while keeping the orchestration core domain-agnostic, so the same machinery drives other domains by swapping prompts.

Mechanistic interpretability. The MI field has produced foundational work on circuits [4], induction heads [16], activation patching [17], and superposition [5]. Recent work has applied MI concepts to biological transformers: Geneformer [2] and scGPT [1] have been probed for gene regulatory network (GRN) inference capabilities. However, these analyses are typically manual, one-shot investigations. MI-Workbench provides

the infrastructure to systematize and scale such investigations.

Scientific workflow systems. Systems like Galaxy [18], Nextflow [19], and Snakemake [20] manage computational biology pipelines. These tools orchestrate deterministic code execution; MI-Workbench orchestrates *reasoning* via LLM agents, where outputs are non-deterministic and require quality assessment.

Materials and methods

MI-Workbench follows a layered architecture (Fig 1) comprising: (i) a REST API layer built on FastAPI with WebSocket support for real-time monitoring; (ii) an orchestration engine that drives the multi-agent loop; (iii) a provider-agnostic adapter layer that interfaces with LLM backends; and (iv) persistence and artifact management via SQLite and a structured file system.

Fig 1. MI-Workbench architecture. The system comprises four layers: an interface layer (CLI, REST API, WebSocket), an orchestration layer (loop engine with state machine, convergence detection, feedback formatting, consensus review, and follow-up generation), a provider-agnostic adapter layer, and a persistence layer.

System architecture

The four layers follow a strict separation of concerns, each isolating one axis of change. The interface layer decouples how a run is launched (CLI, HTTP, or a monitoring socket) from what it does. The orchestration layer holds all coordination logic—graph compilation, sequencing, review merging, convergence, and follow-up generation—in one place, which is what makes control flow auditable (a single reader can follow an entire run) and is the layer this paper evaluates. The adapter layer is a deliberate anti-corruption boundary: because every backend is reached through one narrow `run/smoke.test/is_available` interface, the orchestration logic never depends on a vendor SDK, and new providers or a zero-cost mock are added without touching the engine. The persistence layer records every artifact, prompt, and run parameter so that any run can be reconstructed and packaged for reproduction. This is the standard layering of a service that must be simultaneously vendor-neutral and reproducible; the novelty is in the orchestration layer, not the tiering itself.

Workspace model

A *workspace* represents an isolated research context with its own configuration, provider settings, budget limits, and git integration mode. Each workspace contains one or more *runs*, where each run executes a loop definition against a specific research task. Runs produce versioned *artifacts* organized by iteration: mechanistic analysis (`MECH.md`), evaluation reports (`EVAL.md`), experimental protocols (`XP.md`), methodology documentation (`METHOD.md`), and knowledge summaries (`KNOWLEDGE.md`).

Data model

The system uses Pydantic [21] for all data validation and serialization. Key models include `RunState` (tracking run lifecycle, iterations, token usage, and cost), `IterationResult` (individual step outcomes with artifacts and feedback), `ReviewResult` (structured critique with severity levels and artifact references), and `LoopDefinition` (graph-based workflow specification).

Graph-based workflow DSL

MI-Workbench defines research workflows as directed graphs where nodes represent agent roles and edges encode execution flow with conditions.

Loop definition

A loop definition consists of *nodes* and *edges* specified in YAML:

```
nodes:
  - id: executor
    role: executor
    prompt_ref: "executor/mi_executor"
  - id: reviewer
    role: reviewer
    prompt_ref: "reviewer/mi_reviewer"
  - id: adversarial
    role: adversarial_reviewer
    prompt_ref: "adversarial_reviewer/adversarial_reviewer"
edges:
  - source: executor
    target: reviewer
```

condition: "always"	139
- source: reviewer	140
target: executor	141
condition: "always"	142
- source: reviewer	143
target: adversarial	144
condition: "every_k:3"	145
	146

Edge conditions147

Three condition types are supported:148

- **always**: the edge is traversed every iteration.149
 - **every_k:N**: the edge is traversed every N -th iteration (e.g., adversarial review every 3rd cycle).150151
 - **on_flag:F**: the edge is traversed only when flag F is set (e.g., **on_flag:high_risk_claims**).152153

Compilation154

The DSL compiler performs a topological walk of the graph, validates structural integrity (no unreachable nodes, no duplicate IDs), and produces an ordered `ExecutionPlan` with a cycle length for infinite-loop repetition. Stop conditions include budget limits (tokens, cost, iterations), convergence detection, grade thresholds, and user-initiated cancellation.

155156157158159

Built-in presets160

Four presets are provided:161

1. **executor_reviewer**: standard executor–reviewer loop with periodic adversarial review.162163
 2. **reviewer_consensus**: three parallel reviewers (standard, adversarial, biological plausibility) with consensus merger.164165
 3. **research_followups**: executor–reviewer loop with automated follow-up proposal generation and optional auto-dispatch.166167

4. **example_custom**: advanced loop with conditional adversarial escalation, knowledge extraction, and graph stability detection.

Multi-agent orchestration

A single orchestrator drives every run as a synchronous loop over the compiled plan (Algorithm 1), issuing one agent step per iteration. This centralization is what makes a run reproducible and auditable: the sequence of agent calls for a given loop and task is fully determined by the graph, and the entire control flow lives in one module a reviewer can read end to end. Concurrency is introduced deliberately and locally—within a consensus step the reviewer panel is dispatched with `asyncio.gather`, and independent runs execute as separate asynchronous tasks (up to a configured ceiling)—so the orchestrator scales across concurrent investigations without sacrificing the determinism of any single one. We quantify both properties in the Results.

Agent roles

MI-Workbench defines nine agent roles (Table 1) that fall into four groups. The **executor** performs analysis and emits artifacts. Three **reviewer lenses**—rigour, adversarial, and biological plausibility—critique that output; they are intentionally distinct rather than redundant, each trained by its prompt on a different failure mode (statistical rigour, adversarial confounds, and domain plausibility, respectively), and we show in the Results that they surface largely non-overlapping critiques. Two **generative** roles (idea generator, automator) propose follow-up experiments and reusable pipelines, and three **dissemination and knowledge** roles (knowledge extractor, publication manager, article publisher) structure findings and mine claims. Only the executor and the reviewer lenses are required for a review loop; the remaining roles are optional nodes a workflow may include. Because roles are graph nodes rather than hard-coded stages, a workflow toggles any role by including or omitting it, and the reviewer panel is resized simply by adding or removing lens nodes.

Execution loop

The core execution loop (Algorithm 1) drives the iterative research process.

Table 1. Agent roles and their research functions.

Role	Function
Executor	Performs MI analysis: attention extraction, ablation studies, activation patching; produces MECH.md, XP.md, METHOD.md
Reviewer	Critiques claim validity, reproducibility, confounders, negative controls, and biological plausibility; produces EVAL.md
Adversarial Reviewer	Red-teams analyses for p-hacking, overclaiming, causal confusion, dataset leakage, and statistical theater
Bio-Plausibility Checker	Validates tissue specificity, pathway consistency, expression level plausibility, and regulatory direction
Idea Generator	Produces prioritized follow-up proposals with scientific value, feasibility, and novelty scores
Knowledge Extractor	Mines claims from artifacts, estimates uncertainty, and generates falsification tests
Automator	Designs reusable analysis pipelines from validated protocols
Publication Manager	Structures findings for manuscript preparation
Article Publisher	Formats results for communication

Algorithm 1 MI-Workbench execution loop

Require: RunState R , LoopDefinition L , Adapter A

```

1:  $P \leftarrow \text{COMPILE}(L)$  ▷ Compile to execution plan
2:  $i \leftarrow 0$ ,  $feedback \leftarrow \emptyset$ 
3: while  $\neg \text{BUDGETEXCEEDED}(R) \wedge \neg \text{CONVERGED}(R) \wedge \neg \text{CANCELLED}(R)$  do
4:    $s \leftarrow P.steps[i \bmod |P|]$ 
5:   if  $\neg \text{SHOULDRUN}(s, R.iteration)$  then
6:      $i \leftarrow i + 1$ ; continue
7:   end if
8:    $prompt \leftarrow \text{BUILDPROMPT}(s, R, feedback)$ 
9:    $result \leftarrow \text{RUNWITHRETRY}(A, prompt)$ 
10:   $result.output \leftarrow \text{STRIPTRACES}(result.output)$ 
11:   $artifacts \leftarrow \text{PARSEARTIFACTS}(result.output, s.role)$ 
12:   $\text{WRITEARTIFACTS}(artifacts, R.iteration)$ 
13:  if  $s.role \in \{reviewer, adversarial\}$  then
14:     $review \leftarrow \text{PARSEREVIEW}(result.output)$ 
15:     $feedback \leftarrow \text{FORMATFEEDBACK}(review)$ 
16:  else
17:     $feedback \leftarrow result.output$ 
18:  end if
19:   $\text{RECORDCONVERGENCE}(R.iteration, result)$ 
20:   $R.iteration \leftarrow R.iteration + 1$ ;  $i \leftarrow i + 1$ 
21: end while
22: return  $R$ 

```

Algorithm 1 shows the ordinary single-agent step for clarity. Two step kinds
specialise it. A *consensus* step (compiled from a workflow whose reviewers fan out to a
merger node) replaces the single RUNWITHRETRY call with a concurrent dispatch of
the whole reviewer panel followed by a merge (described below). When verified

execution is enabled, an executor step additionally runs the analysis code it emitted and
appends the captured results to the feedback before the reviewers see it.

Stopping and iteration budget

A run continues until the first of four conditions holds: the iteration ceiling
(`max_iterations`, default 50) is reached; the token budget (default 500,000) or cost
budget is exceeded; convergence is detected; or the user cancels. These bounds cap the
number of feedback rounds; a typical review loop converges well before the ceiling
(Results).

State machine

Run lifecycle is managed by a finite state machine with states {Pending, Running,
Paused, Stopped, Completed, Failed} and well-defined transitions. Paused runs can be
resumed with full state reconstruction from the database and artifact filesystem. A
crash recovery module detects interrupted runs on server restart, transitioning Running
states with existing iterations to Paused for resumability.

Convergence detection

After a minimum of four iterations, three convergence signals are monitored over a
sliding window of w reviews (default $w=3$):

1. **Grade stability:** the reviewer grade has remained constant for w consecutive
reviews.
2. **Zero critical issues:** no Critical or High severity critiques for w reviews.
3. **Output similarity:** Jaccard word-level similarity between consecutive executor
outputs exceeds threshold τ (default 0.9) for w iterations.

Any single signal triggers loop termination, preventing waste of compute on converged
analyses. When the consensus advancement gate is enabled, convergence is suppressed
while any Critical critique remains unresolved, so the loop does not declare success over
an open critical issue.

Retry logic

226

Adapter calls are wrapped in exponential backoff retry with jitter. Transient errors (rate limits, timeouts, connection failures) trigger retry with delay $d_k = d_0 \cdot 2^k \cdot (1 + \epsilon)$, where $\epsilon \sim \mathcal{U}(-0.25, 0.25)$. Permanent errors (authentication failures, invalid requests) fail immediately.

227

228

229

230

Consensus review mechanism

231

A central mechanism of MI-Workbench is the *consensus reviewer*, which replaces single-reviewer assessment with a panel and a defined aggregation rule.

232

233

Panel

234

The panel is a configurable list of reviewer lenses declared in the workflow graph; the default MI panel has three—rigour, adversarial, and biological plausibility—but a workflow may use fewer or more, and the lenses are domain-configurable through their prompts. Within a consensus step the panel is dispatched concurrently over the current executor artifact, so panel latency is that of the slowest lens rather than their sum.

235

236

237

238

239

Critique merging

240

The merge operates in four phases. **Phase 1 (parsing):** each lens’s free text is parsed into structured `ReviewCritique` objects using multi-format detection—bracket (`[SEVERITY] description`), Markdown (`**Severity: X** --- description`), and structured (`Severity: X, Category: Y, Description: Z`). **Phase 2 (deduplication):** critiques are grouped by Jaccard word-overlap similarity,

241

242

243

244

245

$$J(c_i, c_j) = \frac{|\text{words}(c_i) \cap \text{words}(c_j)|}{|\text{words}(c_i) \cup \text{words}(c_j)|}, \quad (1)$$

with pairs satisfying $J(c_i, c_j) \geq \tau$ (default $\tau=0.5$) merged into one group. **Phase 3 (agreement-based escalation):** each group takes the maximum severity present; if at least k distinct lenses raised the group (the *consensus threshold*, default $k=2$), its severity is escalated one level (capped at Critical), encoding the intuition that inter-lens agreement signals importance. **Phase 4 (deterministic ranking):** groups are ordered

246

247

248

249

250

by severity, then by an optional per-role weight, then by the number of contributing
lenses, with the critique text as a final tie-break so the ranking never depends on parse
or dispatch order.

Consensus definition, gating, and deadlock

“Consensus” here is thus an explicit aggregation—dedup, agreement-weighted severity,
and deterministic ranking—rather than a vote, and the loop advances every iteration by
default. When stricter behaviour is wanted, an optional *advancement gate* is the
deadlock policy: while the merged result contains an unresolved Critical critique, the
run is not allowed to converge or complete and instead keeps revising, bounded by the
iteration ceiling so a genuinely irreconcilable panel terminates rather than looping
forever. Ties are broken deterministically as above, so a split panel produces a stable,
reproducible ordering rather than an undefined one.

Grade computation

An overall grade is computed from the merged critique list: ≥ 2 Critical $\Rightarrow$ F; ≥ 1
Critical $\Rightarrow$ D; ≥ 3 High $\Rightarrow$ D; ≥ 1 High $\Rightarrow$ C; ≥ 1 Medium $\Rightarrow$ B; otherwise A.

Structured feedback and output cleaning

Artifact-aware feedback

A key finding from initial deployments was that generic feedback (“Previous iteration
output: ... Continue with: task”) produced shallow revisions. We introduced
artifact-aware feedback with two innovations. First, **reference inference**: when parsing
reviewer critiques, the system infers artifact references (e.g., `PROTOCOL.md`) and section
references (e.g., “Section 2.1 Controls”) from the critique text using pattern matching
against known artifact names and section header patterns. These references are
attached to each `ReviewCritique` as `artifact_ref` and `section_ref` fields. Second,
revision framing: instead of the generic “Continue with: task” prompt, revision
iterations use structured framing:

```
=== REVISION REQUEST (Iteration N) ===  
You produced artifacts in the previous iteration.  
The reviewer has identified specific issues that MUST be addressed.
```

```

=== REVIEWER FEEDBACK (Grade: B) ===

REQUIRED FIXES (must address):
1. [CRITICAL | stats | EVAL.md Section 3.2] p-value without effect size
2. [HIGH | reproducibility | PROTOCOL.md Section 2.1] Missing dataset URLs

SUGGESTED IMPROVEMENTS:
3. [MEDIUM | general] Consider cell-type decomposition

=== END FEEDBACK ===

TASK: Revise your artifacts to address ALL required fixes above.
For each fix, make the specific change in the referenced artifact.
Do NOT rewrite from scratch -- make targeted revisions.

```

Output cleaning pipeline

LLM outputs frequently contain “thinking traces”—narration about the model’s intended actions (e.g., “I will read the file...”, “Let me check the data...”) that contaminate research artifacts. The output cleaning module removes these traces using regex-based line filtering:

- Forward-looking narration: “I will...”, “I’ll...”, “Let me...”, “I need to...”, “I’m going to...”.
- Tool-call narration: “I will read/search/check/verify...the...”.
- Sequencing prefixes: “First, I will...”, “Next, I’ll...”.
- Meta-commentary: “The following is...”.

The cleaner preserves code blocks (tracking ‘‘ fences), markdown headers, and substantive past-tense narration (“I have completed the analysis...”). Excessive blank lines left by removals are collapsed. Cleaning is applied immediately after adapter output, before any downstream consumer (artifact parser, feedback formatter, consensus merger).

Severity stratification and lens weighting

Formatted feedback is stratified by severity—Critical and High critiques become *required fixes* the executor must address, while Medium and below become *suggested improvements*—so revision effort concentrates on what matters. Feedback is uniform across lenses by default; when a deployment trusts one lens more than another (for example, weighting the biological-plausibility lens for a tissue-specific study), an optional per-role weight biases the ranking so that lens’s critiques appear earlier in the required-fixes list without changing which critiques are reported.

Verified execution

By default an executor produces an analysis *plan*; a plan that is never run cannot be checked. When `code_execution_enabled` is set, MI-Workbench extracts the fenced Python blocks from an executor artifact and runs each in an isolated subprocess—a separate `python -I` interpreter in a fresh temporary directory, with a wall-clock timeout, captured stdout/stderr, and an output-size cap—then appends a concise execution report to the feedback stream so the reviewer panel critiques executed results rather than an untested plan. Execution is opt-in and deliberately bounded: it verifies deterministic, self-contained snippets and is not a general sandbox for adversarial code, which would require an operating-system-level jail around the process. Even so, it closes the loop from proposed analysis to observed result and to critique of that result.

Provider-agnostic adapter layer

MI-Workbench defines a `BaseAdapter` abstract interface with three methods: `run(request) → result`, `smoke_test() → status`, and `is_available() → bool`. Four implementations are provided (Table 2).

The mock adapter deserves special mention: it produces realistic MI research outputs with role detection from prompt content, grade progression ($C \rightarrow B \rightarrow B+ \rightarrow A-$), and synthetic metrics (AUROC, correlation, validation splits) with diminishing variation across iterations. This enables rapid prototyping and comprehensive testing without LLM API costs. The adapter registry provides singleton access and availability discovery, enabling the API and CLI to dynamically list available providers and their

Table 2. Adapter implementations and their CLI invocation patterns.

Adapter	CLI binary	Invocation
Claude Code	claude	claude -p --output-format json --max-turns 1 --system-prompt \$SP via stdin
Codex CLI	codex	codex exec --full-auto --skip-git-repo-check - via stdin
Gemini CLI	gemini	gemini --yolo -p \$PROMPT
Mock	—	Synthetic role-aware responses with config- urable grade progression

status. 341

Knowledge extraction and claim management 342

MI-Workbench includes a knowledge extraction pipeline that mines structured claims 343
from research artifacts. **Claim detection:** claims are detected via pattern matching on 344
phrases such as “we found that...”, “results show...”, “AUROC = X”, and “ $p < 0.05$ ”. 345
Detected claims are classified as *empirical*, *methodological*, *interpretive*, or *comparative*. 346
Uncertainty estimation: each claim receives an uncertainty score $u \in [0, 1]$ (baseline 347
0.5) adjusted by strong evidence ($p < 0.001$: $u \leftarrow u - 0.2$), confidence intervals present 348
($u \leftarrow u - 0.1$), hedging language such as “suggests” or “may” ($u \leftarrow u + 0.1$), and large 349
sample $n \geq 1000$ ($u \leftarrow u - 0.1$). **Falsification test generation:** for each claim type, 350
the system auto-generates falsification suggestions—AUROC claims prompt stronger 351
baselines (mutual information, variance-based features), layer-specific claims prompt 352
split-sample validation, attention claims prompt residualization tests, and statistical 353
claims prompt stringent multiple testing correction. **Claim graph:** claims are stored in 354
a relational database with support for inter-claim relationships (supports, contradicts, 355
extends) and evidence linking; a graph API enables visualization of the claim network, 356
supporting identification of weakly-supported claims and contradictions. 357

Reproducibility infrastructure 358

Artifact versioning 359

Each iteration writes artifacts to a structured directory: 360

```
workspace/runs/{run_id}/iter_0001/MECH.md
workspace/runs/{run_id}/iter_0001/EVAL.md
```

361
362
363

```
workspace/runs/{run_id}/iter_0002/MECH.md    (revised)
workspace/runs/{run_id}/run_meta.json
```

This provides full provenance: every revision of every artifact is preserved with its iteration context.

Git integration

Four git modes are supported: `none` (no version control), `commit_per_run` (single commit after run completion), `commit_per_iteration` (commit after each iteration), and `branch_per_run` (create a dedicated branch `run/{run_id}`).

Reproducibility packages

The `repropack` generator creates self-contained ZIP archives containing all artifacts from the run (organized by iteration); `run_meta.json` with run configuration, timing, and cost; `prompts_used.json` with resolved prompt templates; `environment.json` with Python version, installed packages, and OS details; `git_state.json` with branch, commit hash, and dirty status; and auto-generated `dataset_card.md`, `reproducibility_checklist.md`, and `README.md`.

Implementation

MI-Workbench is implemented in Python 3.11 using FastAPI for the REST API, aiosqlite for async database operations, Click for the CLI, and Pydantic for data validation (Table 3).

Table 3. Codebase statistics.

Metric	Value
Backend source files	60+
Backend lines of code	~9,200
Test files	29
Total test functions	370+
Test lines of code	~6,300
Loop preset definitions	4
Prompt templates	9
API endpoints	25+
Supported providers	4
Database tables	7

Key implementation decisions include: *async throughout*, in which all I/O operations (database, adapter calls, file writes) use `async/await` for concurrency; *SQLite with WAL mode*, where write-ahead logging enables concurrent reads during long-running writes, validated under stress testing with 10 concurrent runs; an *event-driven architecture*, in which the engine emits events (`iteration_started`, `iteration_completed`, `budget_exceeded`, etc.) that are broadcast to WebSocket clients for real-time monitoring; and *crash recovery*, in which interrupted runs are detected on server restart and transitioned to resumable states.

Results

We validated MI-Workbench on a substantive MI question: *do attention heads in Geneformer encode causal gene regulatory relationships, or do they merely reflect co-expression patterns?*

Experimental setup

We used the Gemini CLI adapter to run end-to-end experiments across three loop presets (Table 4).

Table 4. Case study runs. All runs used the Gemini CLI adapter with OAuth authentication.

Run	Preset	Iterations	Task summary
1–3	executor_reviewer	1	AUROC interpretation, encoding effects, ablation analysis
4	executor_reviewer	5	Experimental protocol design
5	research_followups	3	Paradox analysis (23× logit perturbation)
6	reviewer_consensus	3	Non-K562 generalization critique
7	executor_reviewer	4	Methods comparison document
8	executor_reviewer	3	Attention–co-expression analysis (validation run)

Key findings from automated analysis

The platform autonomously identified three primary confounders explaining the attention–co-expression correlation. **(1) MLM objective confounder:** the executor agent correctly identified that Geneformer’s masked language modeling objective incentivizes attending to co-expression cliques rather than causal regulators, because

co-expressed genes are the most statistically powerful predictors of masked token identity. **(2) Rank-value encoding artifacts:** the system identified that Geneformer’s rank-value tokenization creates an inductive bias where positional proximity in the sequence corresponds to expression magnitude similarity, producing false-positive attention between co-abundant genes (homophily) rather than regulatory links. **(3) Attention symmetry:** standard dot-product attention $\text{softmax}(QK^\top)$ is inherently bidirectional unless the model learns distinct query/key projections; the executor noted that $A_{ij} \approx A_{ji}$ fails to capture the directional nature of gene regulation (TF $\rightarrow$ Target).

Iterative refinement quality

The structured feedback mechanism produced measurable quality improvements across iterations. In iteration 1, the executor produced an initial analysis with three confounders and a protocol outline with null hypotheses. In iteration 2, the reviewer identified missing controls—a cell-type composition confounder, an unspecified dataset, and missing BH-FDR correction details. In iteration 3, the executor revision added a within-cluster partial correlation control, specified the Genecorpus-30M human bone marrow subset, defined a mean rank baseline, and set a random seed. Critically, the revision iteration produced *targeted changes* to the specific artifacts and sections referenced in the feedback, rather than rewriting from scratch—a direct result of the artifact-aware feedback mechanism.

Output quality assessment

Post-hoc validation by domain experts confirmed that all three identified confounders are well-documented in the literature [2, 7]; that the proposed null hypotheses (H_0^1 : co-expression dominance, H_0^2 : attention symmetry, H_0^3 : degree bias) are scientifically appropriate; that statistical framework choices (AUPRC over AUROC for sparse networks, BH-FDR correction) reflect current best practices; and that the output contained no thinking traces (validated by comparison with pre-cleaning runs where traces contaminated 4/4 artifact files).

Orchestration evaluation

The case study exercises one task; to characterise the orchestrator independently of any single question, we run a set of controlled experiments. Because a full LLM run is stochastic and costly, these use a deterministic mock adapter that emits, per reviewer lens, a fixed critique set with a small, controlled overlap; this isolates the *mechanism* and its *scaling* reproducibly, and we then corroborate the panel with a live Gemini backend. The mock experiments make no claim about research quality—that rests on the case study and the live corroboration.

Does a panel add over a single reviewer? We vary the panel size and measure how many distinct critiques survive merging (Table 5). A single rigour reviewer surfaces three issues; adding the adversarial lens raises this to five and, because both lenses independently flag the missing-effect-size issue, agreement escalates it to Critical (grade F); adding the biological-plausibility lens raises coverage to seven. Coverage grows monotonically with panel size, and agreement upgrades severity exactly when independent lenses concur—evidence that a panel is not redundant with a single well-prompted reviewer.

Table 5. Reviewer-panel ablation (deterministic harness, fixed quality tier). Distinct critiques surviving the merge, the number raised by more than one lens, and the resulting grade, as the panel grows.

Panel	Lenses	Distinct critiques	Multi-lens	Grade
Single	1	3	0	D
Pair	2	5	1	F
Full	3	7	1	F

Are the lenses complementary or redundant? Treating each lens’s critique set as a bag of issues, the pairwise Jaccard overlap is 0.20 for rigour–adversarial and 0.00 for both pairings involving biological plausibility; the union of all three lenses (seven distinct issues) is nearly the sum of their individual counts (eight), so the lenses overlap little and the biology lens in particular contributes issues the others never raise. The roles are complementary by construction, not duplicative.

Convergence. Over 20 independent runs of the consensus loop, every run terminated by convergence rather than the iteration ceiling, at a median of 8 iterations, each time via the grade-stabilization signal. The loop reliably detects when further iteration is unproductive.

Load and scalability. Although each run is coordinated centrally, independent runs execute concurrently. Driving 1 to 20 concurrent runs through the orchestrator (Fig 2), aggregate throughput rises from about 110 to 376 iterations/s while p95 per-run latency grows sub-proportionally (from 55 ms to 314 ms at 20 $\times$ concurrency), and a database-integrity check passes after every level. The centralized coordinator therefore scales across concurrent investigations, with tail latency degrading gracefully rather than collapsing.

Fig 2. Orchestrator load scaling. (a) Aggregate throughput and (b) p95 per-run latency as the number of concurrent runs increases from 1 to 20. Throughput rises with concurrency while tail latency grows sub-proportionally.

Live-backend corroboration. To confirm the wired panel behaves as characterised with a real model, we ran the three-lens panel once over a fixed Geneformer analysis artifact using the Gemini backend. All three lenses returned; the rigour lens alone produced 12 critiques, and the full panel produced 23 distinct merged critiques—roughly double the coverage—consistent with the ablation. Notably, verbatim Jaccard deduplication rarely merged critiques across lenses on free-text output (the lenses phrase overlapping concerns differently), which both underlines the coverage benefit of a panel and motivates semantic, rather than lexical, deduplication as a refinement.

Discussion and conclusion

MI-Workbench shows that a centrally orchestrated team of LLM agents can carry out mechanistic-interpretability investigations that are rigorous, reproducible, and inspectable. The evaluation supports its core design choices: a reviewer panel surfaces strictly more, and complementary, critiques than a single reviewer; agreement between lenses escalates severity where it should; the loop reliably detects convergence; and the centralized coordinator scales across concurrent runs with graceful tail-latency growth. The provider-agnostic design insulates workflows from any one vendor, and the accompanying test suite exercises the orchestration, consensus, feedback, and execution paths.

We deliberately trade agent autonomy for a deterministic, auditable control flow, which is the right trade for automated scientific critique: a run’s sequence of agent calls

follows from its workflow graph, every artifact and prompt is persisted, and any run can
be repackaged for independent reproduction. The domain specificity lives in swappable
prompts and configurable reviewer lenses rather than in the engine, so the same
coordinator drives other domains and other panel compositions; the evaluation
experiments are themselves task-independent evidence of this separation.

Several limitations remain. Verified execution is opt-in and intentionally
bounded—it runs deterministic, self-contained snippets in an isolated subprocess but is
not a hardened sandbox for adversarial code, which would require an
operating-system-level jail. Critique deduplication is lexical (word-level Jaccard); as the
live-backend corroboration shows, real reviewers phrase overlapping concerns differently,
so semantic deduplication via embeddings would merge and escalate agreement more
faithfully and would also sharpen the convergence signal. Cross-provider cost
accounting depends on adapter-level reporting and is not yet standardized. Finally,
coordination is centralized: this is a feature for reproducibility but caps single-run
parallelism at the reviewer panel, and there is no mechanism for mid-loop human
redirection beyond the follow-up approval policy. Natural extensions include semantic
merging and convergence, multi-model ensemble execution that runs a task across
several backends and compares outputs, integration with laboratory-automation systems
for wet-lab validation, and richer human-in-the-loop control.

In sum, MI-Workbench contributes a graph-based workflow DSL, a concurrent
consensus-review mechanism with a defined aggregation and gating semantics,
artifact-aware structured feedback, sandboxed verified execution, and a
provider-agnostic architecture, evaluated with both controlled orchestration experiments
and a live-backend case study. It provides a foundation for scaling mechanistic
interpretability of biological foundation models beyond the capacity of individual
researchers.

Supporting information

S1 Fig. MI-Workbench architecture diagram (high resolution).

Full-resolution version of Fig 1, provided as a standalone vector source (**Fig1.tex**) for
reproduction.

S1 File. Source code and reproducibility package. The complete MI-Workbench implementation (backend, CLI, frontend, prompt templates, loop presets, and test suite) is available at <https://github.com/Biodyn-AI/mi-workbench>. Reproducibility packages for the case study runs are generated by the `repropack` module and bundle all artifacts, resolved prompts, environment metadata, and git provenance.

S2 File. Evaluation harness and results. The orchestration experiments (panel ablation, reviewer-lens overlap, convergence profile, and concurrent-run load benchmark) are implemented in the `backend/analysis/experiments.py` module and reproduced by `scripts/run_experiments.py`, which writes the machine-readable results consumed by Table 5 and Fig 2. The live-backend corroboration is reproduced by `scripts/real_consensus_probe.py`.

Acknowledgments

The author thanks the developers of the open-source tools on which MI-Workbench is built, including FastAPI, Pydantic, and the Claude Code, OpenAI Codex, and Google Gemini command-line interfaces.

References

1. Cui H, Wang C, Maan H, Pang K, Luo F, Duan N, et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*. 2024;21(8):1470–1480.
2. Theodoris CV, Xiao L, Chopra A, Chaffin MD, Al Sayed ZR, Hill MC, et al. Transfer learning enables predictions in network biology. *Nature*. 2023;618(7965):616–624.
3. Yang F, Wang W, Wang F, Fang Y, Tang D, Huang J, et al. scBERT as a large-scale pretrained deep language model for cell type annotation of single-cell RNA-seq data. *Nature Machine Intelligence*. 2022;4(10):852–866.
4. Olah C, Cammarata N, Schubert L, Goh G, Petrov M, Carter S. Zoom in: an introduction to circuits. *Distill*. 2020;5(3):e00024.001.

5. Elhage N, Hume T, Olsson C, Schiefer N, Henighan T, Kravec S, et al. Toy models of superposition. *Transformer Circuits Thread*. 2022;.
6. Conmy A, Mavor-Parker AN, Lynch A, Heimersheim S, Garriga-Alonso A. Towards automated circuit discovery for mechanistic interpretability. *Advances in Neural Information Processing Systems*. 2023;36.
7. Pratapa A, Jalihal AP, Law JN, Bharadwaj A, Murali TM. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nature Methods*. 2020;17(2):147–154.
8. Romera-Paredes B, Barekatin M, Novikov A, Balog M, Kumar MP, Dupont E, et al. Mathematical discoveries from program search with large language models. *Nature*. 2024;625(7995):468–475.
9. Boiko DA, MacKnight R, Kline B, Gomes G. Autonomous chemical research with large language models. *Nature*. 2023;624(7992):570–578.
10. Bran AM, Cox S, Schilter O, Baldassari C, White AD, Schwaller P. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*. 2024;6(5):525–535.
11. Ghafarollahi A, Buehler MJ. SciAgents: automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*. 2025;37(22):2413523.
12. Li G, Hammoud HAAK, Itani H, Khizbullin D, Ghanem B. CAMEL: communicative agents for “mind” exploration of large language model society. *Advances in Neural Information Processing Systems*. 2023;36.
13. Wu Q, Bansal G, Zhang J, Wu Y, Li B, Zhu E, et al. AutoGen: enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:230808155*. 2023;.
14. Qian C, Liu W, Liu H, Chen N, Dang Y, Li J, et al. ChatDev: communicative agents for software development. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*; 2024.

15. Hong S, Zhuge M, Chen J, Zheng X, Cheng Y, Zhang C, et al. MetaGPT: meta programming for a multi-agent collaborative framework. In: The Twelfth International Conference on Learning Representations (ICLR); 2024.
16. Olsson C, Elhage N, Nanda N, Joseph N, DasSarma N, Henighan T, et al. In-context learning and induction heads. Transformer Circuits Thread. 2022;.
17. Meng K, Bau D, Andonian A, Belinkov Y. Locating and editing factual associations in GPT. Advances in Neural Information Processing Systems. 2022;35.
18. Afgan E, Baker D, Batut B, van den Beek M, Bouvier D, Čech M, et al. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. Nucleic Acids Research. 2018;46(W1):W537–W544.
19. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. Nature Biotechnology. 2017;35(4):316–319.
20. Mölder F, Jablonski KP, Letcher B, Hall MB, Tomkins-Tinch CH, Sochat V, et al. Sustainable data analysis with Snakemake. F1000Research. 2021;10:33.
21. Pydantic. Pydantic: data validation using Python type hints; 2024. <https://docs.pydantic.dev/>.